UNIVERSIDAD TORCUATO DI TELLA

TD7: Ingeniería de Datos

SQL: Parte III

Dataset para la clase de hoy

Cláusula WITH

- La cláusula WITH nos permite definir una tabla temporal para ser utilizada en una consulta.
- Esta tabla nunca se almacenará. Sólo existirá durante el cálculo de la consulta.

Sintaxis:

```
WITH nombre_tabla_temp AS (SELECT …)
SELECT A1, A2, …, An
        FROM T1, T2, …, Tn
        [ WHERE condición ];
```

Subconsulta auxiliar

Consulta principal

Cláusula WITH: ejercicio

7.Utilizando una cláusula WITH, listar nombres de playlists que tienen tracks que aparecen en las facturas de menor importe.

```
WITH facturasConMenorImporte AS
     (Select invoice_id
             FROM invoice
     Where total = (Select Min(total) from invoice))

Select DISTINCT p.name
     From playlist as p
     INNER JOIN playlist_track as pt ON pt. playlist_id = p.playlist_id
     INNER JOIN track as t ON pt.track_id = t.track_id
     INNER JOIN invoice_line as il ON il.track_id = t.track_id
     INNER JOIN facturasConMenorImporte as f ON f.invoice_id = il.invoice_id
```

Consultas anidadas, menú completo

- Existen varias formas de anidar una consulta en el WHERE:
  - WHERE valor = (SELECT…) → solo funciona si la subconsulta devuelve una columna con un valor
  - WHERE valor **IN|NOT IN** (SELECT…) → funciona si la subconsulta devuelve sólo 1 columna o si "valor" es una tupla y la consulta devuelve un tupla similar.
  - WHERE valor **>= ALL|SOME** (SELECT…) → funciona si la subconsulta devuelve sólo 1 columna
  - WHERE **EXISTS|NOT EXISTS** (SELECT…) → funciona siempre
- Cuando la consulta anidada utiliza columnas de una fila de alguna tabla de la consulta principal, se dice que las consultas de adentro y la de afuera están **correlacionadas**.
  - Esto implica que la consulta de adentro deberá ejecutarse una vez por cada fila de la consulta de afuera.

Consultas anidadas, menú completo: ejercicio

8. Usando consultas anidadas y la cláusula NOT EXIST seleccione nombre del track y composer de los tracks que nunca fueron facturados.

```
SELECT  track.name, track.composer
     FROM track
     WHERE NOT EXISTS (
              SELECT *
              FROM invoice_line
              WHERE invoice_line.track_id = track.track_id )
```

Operaciones de conjuntos en SQL

Ejemplo

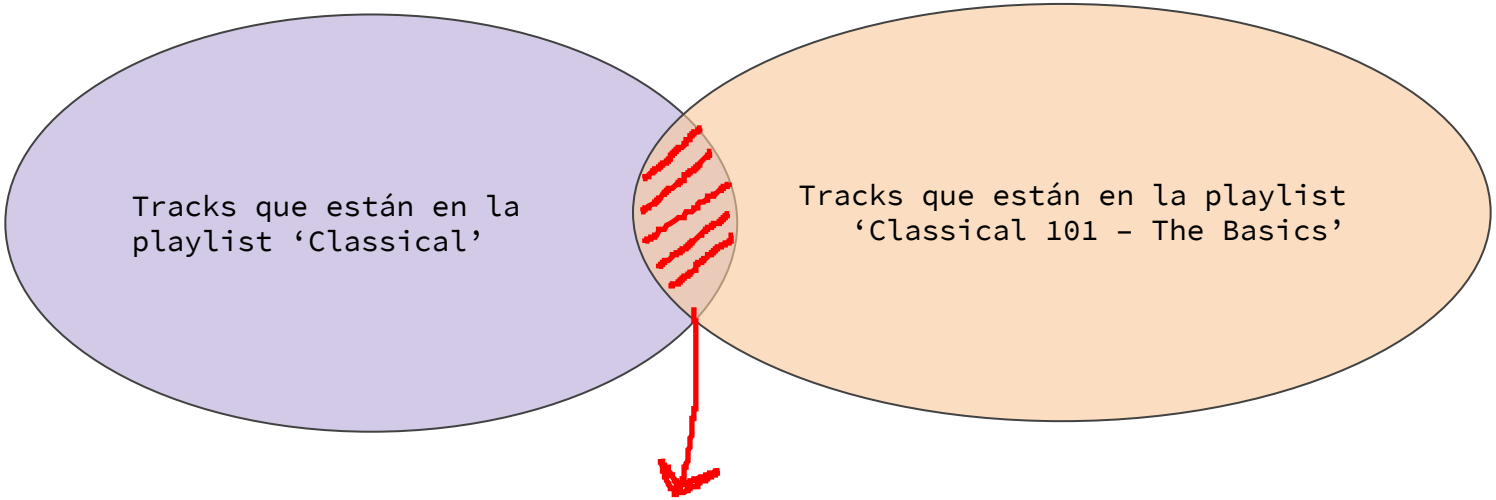
Supongamos que queremos encontrar los nombres de los tracks que están tanto en la playlist de nombre ‘Classical’ como en la de nombre ‘Classical 101 - The Basics’. La siguiente podría ser una forma de resolverlo:

```
Select t.name, p1.name, p2.name
from track t, playlist p1, playlist_track pt1, playlist_track pt2,playlist p2
        where pt1.track_id = pt2.track_id
                and t.track_id = pt1.track_id
                and pt1.playlist_id = p1.playlist_id
                and pt2.playlist_id = p2.playlist_id
                and p1.name = 'Classical’
                and p2.name = 'Classical 101 - The Basics'
```

Complicadísimo e ilegible

Ejemplo

- Sin embargo, también podríamos resolverlo de la siguiente forma:



- Es decir, con una **intersección** de conjuntos.

Operaciones de conjuntos

- SQL permite calcular la unión, intersección o diferencia del resultado de dos consultas SQL independientes.
- El resultado es análogo a lo que ocurría en el álgebra relacional con ∪, ∩, -.
- Al igual que en álgebra, se requiere **compatibilidad de tipos** entre las relaciones.
- La sintaxis es:

```
( SELECT … consulta SQL 1 )

    UNION | INTERSECT | EXCEPT

( SELECT … consulta SQL 2 )
```

Operaciones de conjuntos: ejercicio

2.Encontrar los nombres de los tracks que están tanto en la playlist de nombre ‘Classical’ como en la de nombre ‘Classical 101 - The Basics’ utilizando operaciones de conjuntos en SQL.

```
( SELECT t.track_id, t.name FROM Track t, Playlist p, playlist_track pt
     WHERE p.playlist_id = pt.playlist_id
AND pt.track_id = t.track_id AND p.name = 'Classical' )
INTERSECT
( SELECT t.track_id, t.name FROM Track t, Playlist p, playlist_track pt
     WHERE p.playlist_id = pt.playlist_id
AND pt.track_id = t.track_id AND p.name = 'Classical 101 - The Basics')
```

Operaciones de conjuntos: ejercicio 2

3.Encontrar los nombres de los tracks que están o bien en la playlist de nombre ‘Music Videos’ o bien en la de nombre ‘On-The-Go 1’ utilizando operaciones de conjuntos en SQL.

```
( SELECT t.track_id, t.name     FROM Track t, Playlist p, Playlist_track pt
              WHERE p.playlist_id = pt.playlist_id
AND pt.track_id = t.track_id AND p.name = 'Music Videos’ )
UNION
( SELECT t.track_id, t.name     FROM Track t, Playlist p, Playlist_track pt
              WHERE p.playlist_id = pt.playlist_id
AND pt.track_id = t.track_id AND p.name = 'On-The-Go 1' )
```

Junta externa (OUTER JOIN)

Ejemplo

- Supongamos ahora que queremos mostrar para cada artista cuántos álbumes tiene de su autoría en nuestra base de datos, pero asegurándonos de no perder a aquellos de los cuáles no tenemos registrado un álbum. ¿Qué ocurriría con la siguiente solución?

```
SELECT a.name, COUNT(*)
        FROM Artist a INNER JOIN Album al
                ON (a.artist_id = al.artist_id)
                GROUP BY a.artist_id, a.name;
```

Con este INNER JOIN nos perdemos a los artistas que no tengan álbumes o álbumes que no tengan artistas.
Agarra los que tengan un artista y álbum asociados.

Ejemplo

- Para asegurarnos de que todos los artistas ‘a’ que aparecen del lado izquierdo de la consulta sobrevivan en el resultado, se puede usar un **LEFT JOIN**:

```
SELECT a.name, COUNT(album_id)
        FROM Artist a LEFT JOIN Album al
                ON (a.artist_id = al.artist_id)
                GROUP BY a.artist_id, a.name;
```

- De esta manera, los artistas que no se vinculan con ningún álbum quedan presentes en el resultado de la junta.

Outer join

- El ***outer join*** o **junta externa** permite juntar dos tablas asegurándonos de que los valores en las filas de ellas siempre aparezcan en el resultado, aún cuando no hayan logrado combinarse con ninguna fila de la otra tabla.
- Viene en 3 variantes:
  - **LEFT OUTER JOIN:** Asegura que las filas de la tabla izquierda siempre estén en el resultado.
  - **RIGHT OUTER JOIN:** Asegura que las filas de la tabla derecha siempre estén en el resultado.
  - **FULL OUTER JOIN:** Asegura que las filas de ambas tablas siempre estén en el resultado.
- Para lograr esto, el OUTER JOIN completa en el resultado las filas que no logró conectar con valores nulos (NULL).

Outer join: sintaxis

```
SELECT A1, A2, …, An
    FROM T1 LEFT|RIGHT|FULL [OUTER] JOIN T2 ON (cond_junta)
    [ WHERE condicion ];
```

Outer join: ejercicio

4.Para cada track de género “Rock And Roll” muestre el nombre del track y la cantidad de invoice_lines en las que aparece, asegurándose de que si un track nunca estuvo en una factura se devuelva cero.

¿Por qué no puede ser COUNT(*)?

```
SELECT t.name, COUNT(i.invoice_line_id)
    FROM Track t INNER JOIN genre g on (t.genre_id = g.genre_id)
LEFT JOIN Invoice_line i ON (t.track_id = i.track_id)
    WHERE g.name = 'Rock And Roll'
    GROUP BY t.name
```

Comportamiento de COUNT()

Comportamiento de COUNT()

- La función de agregación COUNT() tiene distinto comportamiento según la forma en que la utilizamos:
  - **COUNT(A):** Cuenta la cantidad de valores no nulos de A
  - **COUNT(DISTINCT A):** Cuenta la cantidad de valores no nulos distintos de A
  - **COUNT(*):** Cuenta la cantidad de filas totales
- En el ejemplo anterior, si hubiéramos utilizado COUNT(*), el COUNT iba a dar uno en los casos en que un track no está asociado a una invoice_line.

Funciones auxiliares

Funciones auxiliares

- Funciones auxiliares para manejo de strings:
  - **SUBSTRING(string FROM start FOR length):** Selecciona un substring desde la posición start y de largo length.
  - **UPPER(string)/LOWER(string):** Convierte el string a mayúsculas/minúsculas.
  - **CHAR_LENGTH(string):** Devuelve la longitud del string.

- **COALESCE(expr1, expr2, …, exprN):** Devuelve para cada fila la primera expresión del listado cuyo valor es no nulo. Ejemplo: COALESCE(domicilio, '<desconocido>')

Otras funciones auxiliares de PostgreSQL: https://www.postgresql.org/docs/8.1/functions.html

Resumen

- Hoy aumentamos aún más nuestro conocimiento del lenguaje SQL para consultar datos, aprendiendo:
  - Incluir consultas anidadas dentro de otras consultas (introducidas con =, o IN|NOT IN, o EXISTS|NOT EXISTS).
  - Utilizar la cláusula WITH para definir tablas temporales durante una consulta.
  - Las operaciones de conjuntos con UNION, INTERSECT y EXCEPT.
  - El concepto de junta externa y su utilidad para encontrar filas que no logran vincularse con ninguna de otra tabla, o para asegurarnos no excluir ciertos datos del resultado cuando hacemos un join.
  - El comportamiento general de la función COUNT().
- Por último, conocimos el comportamiento de las funciones de ventana.